

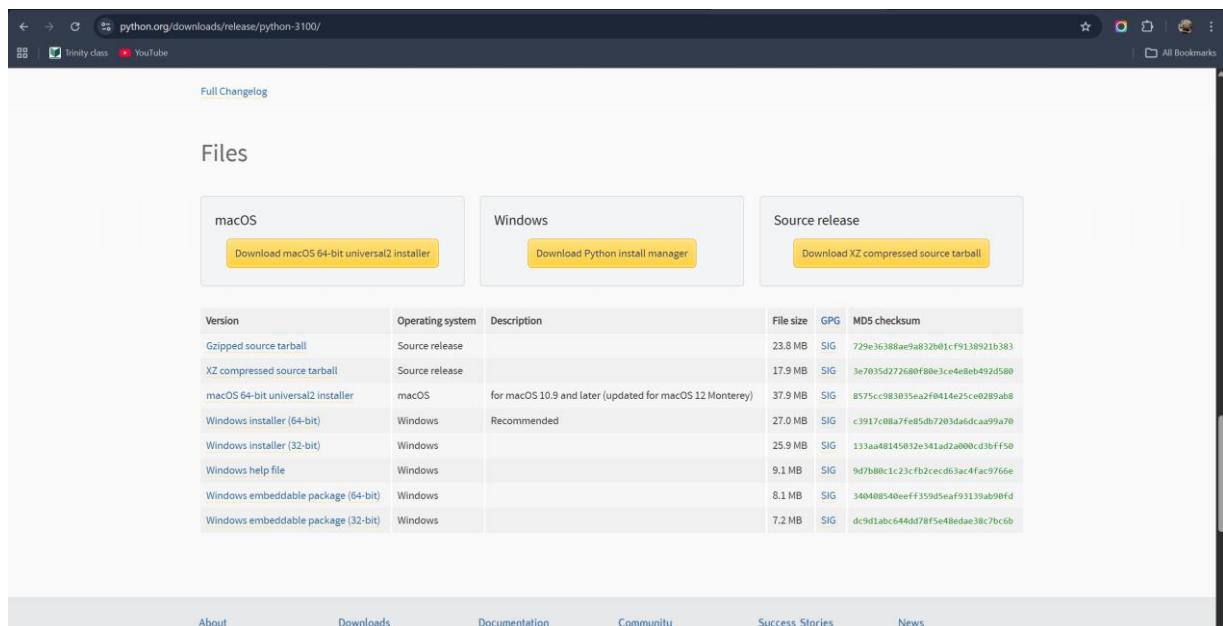
Installation & User Guide

Comparative Evaluation of Vision-Based Finger Tracking for Piano Interaction

1. System Requirements

Before installing, confirm your system meets the following prerequisites:

- Python 3.10 (required — MediaPipe 0.10.x is incompatible with Python 3.11+)
- Windows 10 / 11 (primary platform; macOS and Linux work with minor path changes)
- Git (for cloning the repository)
- A USB webcam or built-in camera
- A MIDI keyboard connected via USB
- ~2 GB free disk space (model weights + Python packages)



The screenshot shows the Python 3.10 download page. It features three main download buttons: 'Download macOS 64-bit universal2 installer', 'Download Python install manager', and 'Download XZ compressed source tarball'. Below these is a table listing various files available for download.

Version	Operating system	Description	File size	GPG	MD5 checksum
Gzipped source tarball	Source release		23.8 MB	SIG	729e36388ae9a832b01cf9138921b383
XZ compressed source tarball	Source release		17.9 MB	SIG	3e7035d272680f88e3c4e48eb492d580
macOS 64-bit universal2 installer	macOS	for macOS 10.9 and later (updated for macOS 12 Monterey)	37.9 MB	SIG	8575cc583835ea2f0414e25ce0289ab8
Windows installer (64-bit)	Windows	Recommended	27.0 MB	SIG	c3917c08a7fe5db7203dadca99a70
Windows installer (32-bit)	Windows		25.9 MB	SIG	133aa48145032e341ad2a000cd3bfff50
Windows help file	Windows		9.1 MB	SIG	9d7b88c1c23cfb2cec063ac4fac9766e
Windows embeddable package (64-bit)	Windows		8.1 MB	SIG	340488540eeff359d5ea93139ab90fd
Windows embeddable package (32-bit)	Windows		7.2 MB	SIG	dc9d1abc6446d78f5e48edae38c7bc6b

Python 3.10 download page at python.org/downloads

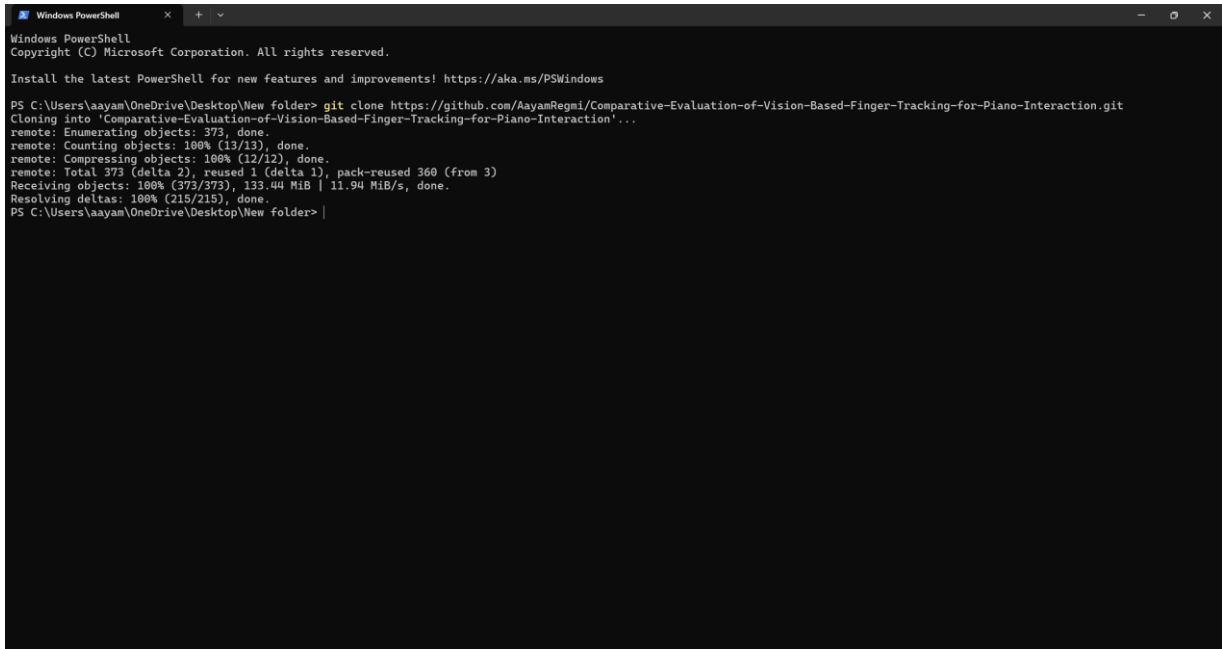
WARNING Do not use Python 3.11 or higher. mediapipe==0.10.11 will fail to install or crash at runtime on anything above 3.10.

2. Step-by-Step Installation

2.1 Clone the Repository

Open a terminal and run:

```
git clone https://github.com/AayamRegmi/Comparative-Evaluation-of-Vision-Based-Finger-Tracking-for-Piano-Interaction.git
cd Comparative-Evaluation-of-Vision-Based-Finger-Tracking-for-Piano-Interaction
```

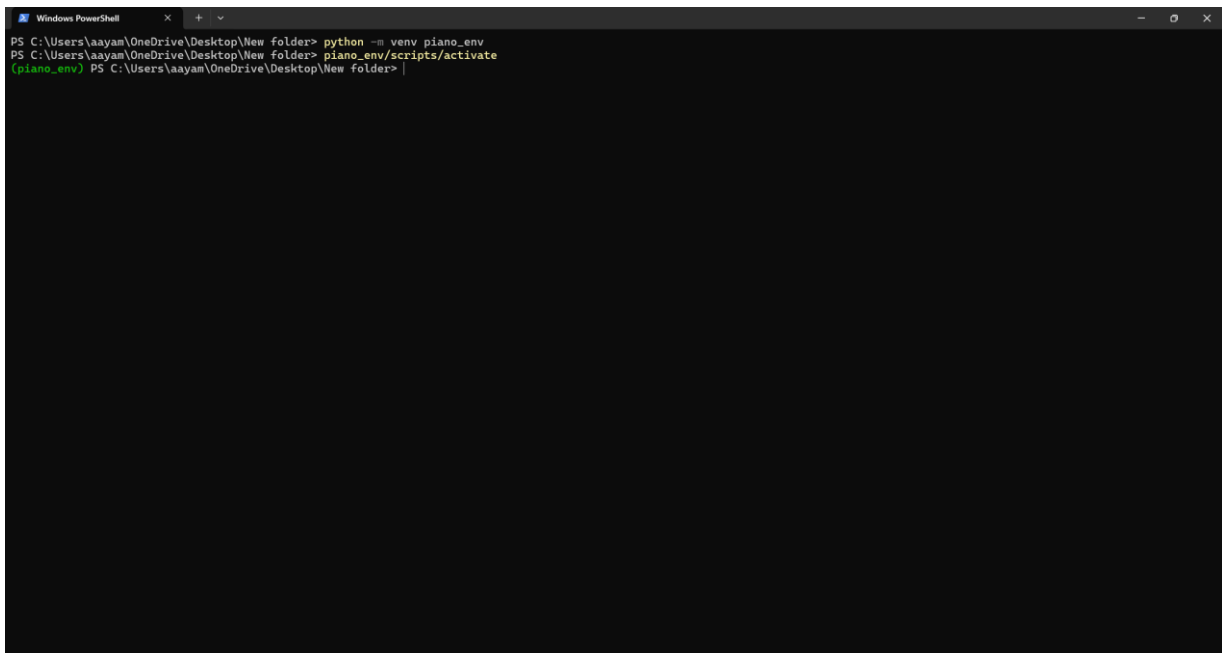
A screenshot of a Windows PowerShell terminal window. The window title is "Windows PowerShell". The output shows the standard PowerShell startup text, followed by a message to install the latest PowerShell. The user then runs the command: `PS C:\Users\Aayam\OneDrive\Desktop\New folder> git clone https://github.com/AayamRegmi/Comparative-Evaluation-of-Vision-Based-Finger-Tracking-for-Piano-Interaction.git`. The terminal displays the progress of the clone operation, including cloning into the specified path, enumerating objects (373), counting objects (100%), compressing objects (100%), and receiving objects (100%). The final output is `PS C:\Users\Aayam\OneDrive\Desktop\New folder> |`.

Terminal window showing successful git clone output

2.2 Create a Virtual Environment

Always use a dedicated virtual environment to avoid package conflicts:

```
python -m venv piano_env
```

A screenshot of a Windows PowerShell terminal window. The window title is "Windows PowerShell". The command prompt shows the user's current directory as "C:\Users\ayam\OneDrive\Desktop\New folder". The user enters the command "python -m venv piano_env", which creates a new virtual environment. The prompt then changes to "(piano_env) PS C:\Users\ayam\OneDrive\Desktop\New folder>". The user enters the command "piano_env/scripts/activate", which activates the virtual environment. The prompt then changes to "(piano_env) PS C:\Users\ayam\OneDrive\Desktop\New folder>".

```
PS C:\Users\ayam\OneDrive\Desktop\New folder> python -m venv piano_env
PS C:\Users\ayam\OneDrive\Desktop\New folder> piano_env/scripts/activate
(piano_env) PS C:\Users\ayam\OneDrive\Desktop\New folder>
```

Terminal showing virtual environment creation (piano_env folder created)

2.3 Activate the Virtual Environment

Windows (Command Prompt / PowerShell):

```
piano_env\Scripts\activate
```

macOS / Linux:

```
source piano_env/bin/activate
```

Your terminal prompt should show (piano_env) once activated.

2.4 Install Python Dependencies

With the virtual environment active, install all required packages from the requirements file:

```
pip install -r requirements.txt
```

This installs: NumPy, Pandas, OpenCV, MediaPipe, Mido, python-rtmidi, Matplotlib, Seaborn, SciPy, and tqdm.

```
Windows PowerShell
[piano_env] PS C:\Users\laayam\OneDrive\Desktop\New folder\Comparative-Evaluation-of-Vision-Based-Finger-Tracking-for-Piano-Interaction> pip install -r requirements.txt
Collecting numpy==1.26.4
  Using cached numpy-1.26.4-cp310-cp310-win_amd64.whl (15.8 MB)
Collecting pandas==2.2.2
  Using cached pandas-2.2.2-cp310-cp310-win_amd64.whl (11.6 MB)
Collecting opencv-python==4.9.0.80
  Using cached opencv_python-4.9.0.80-cp37-abi3-win_amd64.whl (38.6 MB)
Collecting mediapipe==0.10.11
  Using cached mediapipe-0.10.11-cp310-cp310-win_amd64.whl (50.8 MB)
Collecting tensorflow==2.15.0
  Using cached tensorflow-2.15.0-cp310-cp310-win_amd64.whl (2.1 kB)
Collecting tensorflow-hub==0.16.1
  Using cached tensorflow_hub-0.16.1-py2.py3-none-any.whl (30 kB)
Collecting mido==1.3.3
  Using cached mido-1.3.3-py3-none-any.whl (54 kB)
Collecting python-rtmidi==1.5.8
  Using cached python_rtmidi-1.5.8-cp310-cp310-win_amd64.whl (132 kB)
Collecting matplotlib==3.8.4
  Using cached matplotlib-3.8.4-cp310-cp310-win_amd64.whl (7.7 MB)
Collecting seaborn==0.13.2
  Using cached seaborn-0.13.2-py3-none-any.whl (294 kB)
Collecting scipy==1.13.1
  Using cached scipy-1.13.1-cp310-cp310-win_amd64.whl (46.2 MB)
Collecting tqdm==4.66.4
  Using cached tqdm-4.66.4-py3-none-any.whl (78 kB)
Collecting python-dateutil>=2.8.2
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Collecting tzdata>=2022.7
  Downloading tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Collecting pytz>=2020.1
  Downloading pytz-2026.2-py2.py3-none-any.whl (510 kB)
Collecting jax
  Downloading jax-0.6.2-py3-none-any.whl (2.7 MB)
Collecting flatbuffers>=2.0
  Using cached flatbuffers-25.12.19-py2.py3-none-any.whl (26 kB)
Collecting protobuf<4,>=3.11
  Using cached protobuf-3.20.3-cp310-cp310-win_amd64.whl (904 kB)
Collecting sounddevice==0.4.4
  Using cached sounddevice-0.5.5-py3-none-win_amd64.whl (365 kB)
Collecting opencv-contrib-python
  Using cached opencv_contrib_python-4.13.0.92-cp37-abi3-win_amd64.whl (46.5 MB)
Collecting absl-py
  Using cached absl_py-2.4.0-py3-none-any.whl (135 kB)
Collecting attrs>=19.1.0
```

Terminal showing pip install completing with no errors

WARNING Windows only — python-rtmidi requires the Microsoft C++ Build Tools to compile. If the install fails with a compiler error, download and install 'Build Tools for Visual Studio' from visualstudio.microsoft.com (select Desktop development with C++), then re-run the pip command.

2.5 Install Additional Packages

Three packages used by analysis and results scripts are not included in requirements.txt and must be installed separately:

```
pip install Pillow python-pptx python-docx
```

Package	Version tested	Used by
Pillow	12.x	live_preview.py – results overlay rendering
python-pptx	1.x	redesign_pptx.py – presentation export
python-docx	any	Documentation helper scripts

2.6 OpenPose Model Files

OpenPose hand tracking runs through OpenCV's DNN module using pre-trained Caffe weights. The model files are already included in the repository under scripts/ — no separate OpenPose installation is needed:

- scripts/openpose_hand.prototxt — network architecture
- scripts/openpose_hand.caffemodel — pre-trained weights (~55 MB)

NOTE If the caffemodel file is missing (e.g. after a shallow clone), run python download_openpose_model.py from the project root to re-download it.

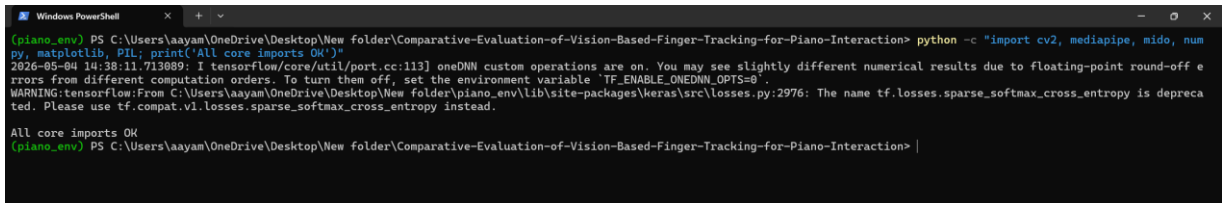
3. Verify the Installation

With the virtual environment active, run the following one-liner to confirm all core libraries import successfully:

```
python -c "import cv2, mediapipe, mido, numpy, matplotlib, PIL; print('All core imports OK')"
```

Expected output:

```
All core imports OK
```



```

(piano_env) PS C:\Users\ayyam\OneDrive\Desktop\New folder\Comparative-Evaluation-of-Vision-Based-Finger-Tracking-for-Piano-Interaction> python -c "import cv2, mediapipe, mido, numpy, matplotlib, PIL; print('All core imports OK')"
2026-05-04 14:38:11.713089: I tensorflow/core/util/port.cc:113] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
WARNING:tensorflow:From C:\Users\ayyam\OneDrive\Desktop\New folder\piano_env\lib\site-packages\keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

All core imports OK
(piano_env) PS C:\Users\ayyam\OneDrive\Desktop\New folder\Comparative-Evaluation-of-Vision-Based-Finger-Tracking-for-Piano-Interaction>

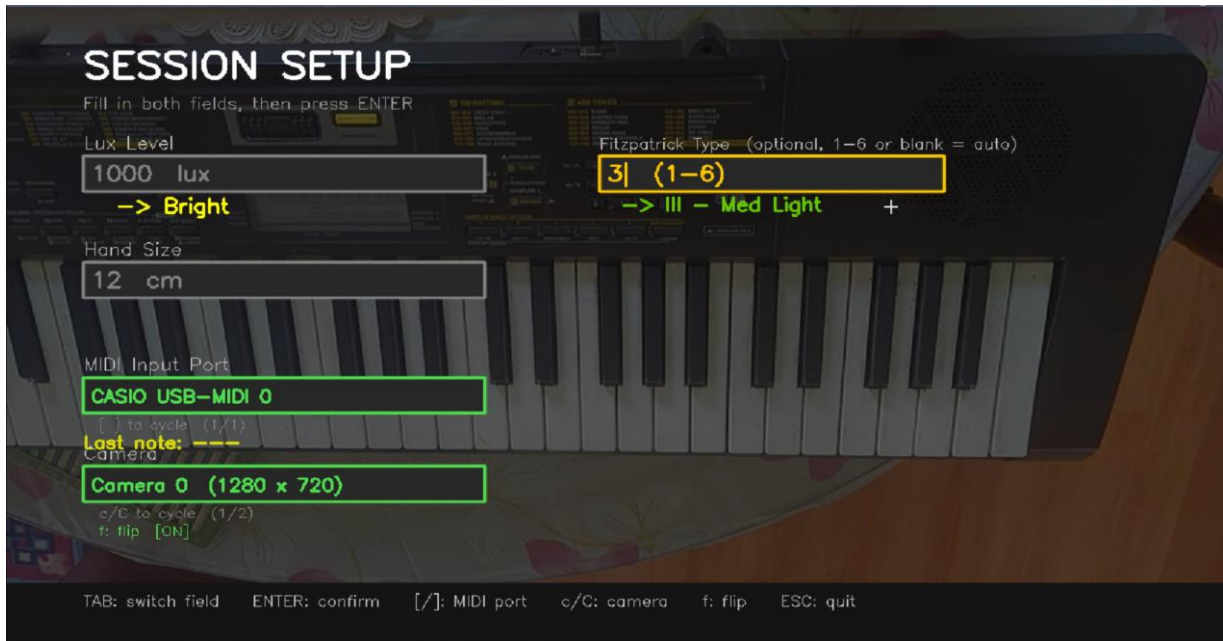
```

Terminal showing 'All core imports OK' with no errors or warnings

4. MIDI Device Setup

The system reads MIDI events in real time to log which keys are pressed during recording and testing sessions.

- Connect the MIDI keyboard to the computer via USB before launching any script.
- Windows may require a driver from the keyboard manufacturer — check the product page if the keyboard is not detected.
- On the setup screen that appears when launching record.py or live_preview.py, the detected MIDI ports are listed.
- Use the [and] keys to cycle between available ports and select the correct one.
- Press a key on the piano to confirm the 'Last note' field updates — this confirms the MIDI connection is working.



Setup screen showing the MIDI port selection box and 'Last note' live readout

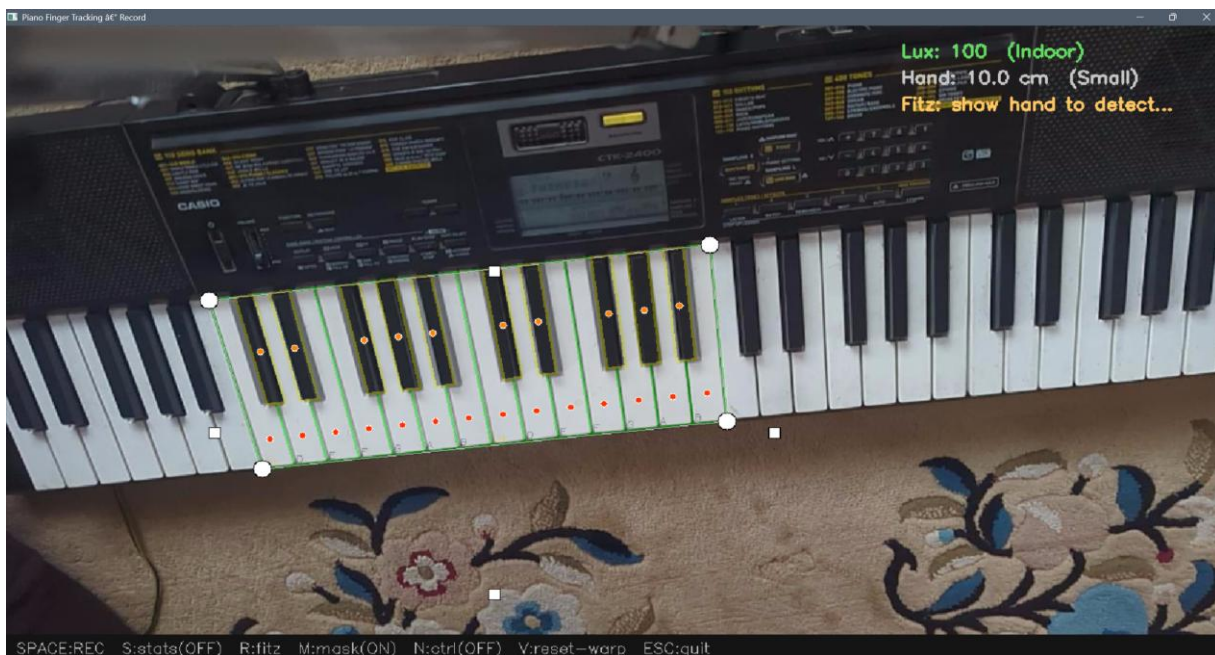
5. Running the Scripts

All scripts are run as Python modules from the project root with the virtual environment active. The table below summarises each script's purpose:

Script	Command	Purpose
key_calibration	<code>python -m scripts.key_calibration</code>	Align the on-screen keyboard overlay to the camera view and save calibration
live_preview	<code>python -m scripts.live_preview</code>	Real-time MPJPE accuracy test – no data written to disk
record	<code>python -m scripts.record</code>	Record a labelled session: video, MIDI, and session metadata
analyse	<code>python -m scripts.analyse</code>	Offline MPJPE analysis of recorded sessions via GUI or CLI
plot_results	<code>python -m scripts.plot_results</code>	Generate all charts and participant reports from analysed data

5.1 Key Calibration (`key_calibration.py`)

Run this first, or whenever the camera position changes. Drag the keyboard overlay to match the physical piano keys in the camera view, then press ENTER to save.



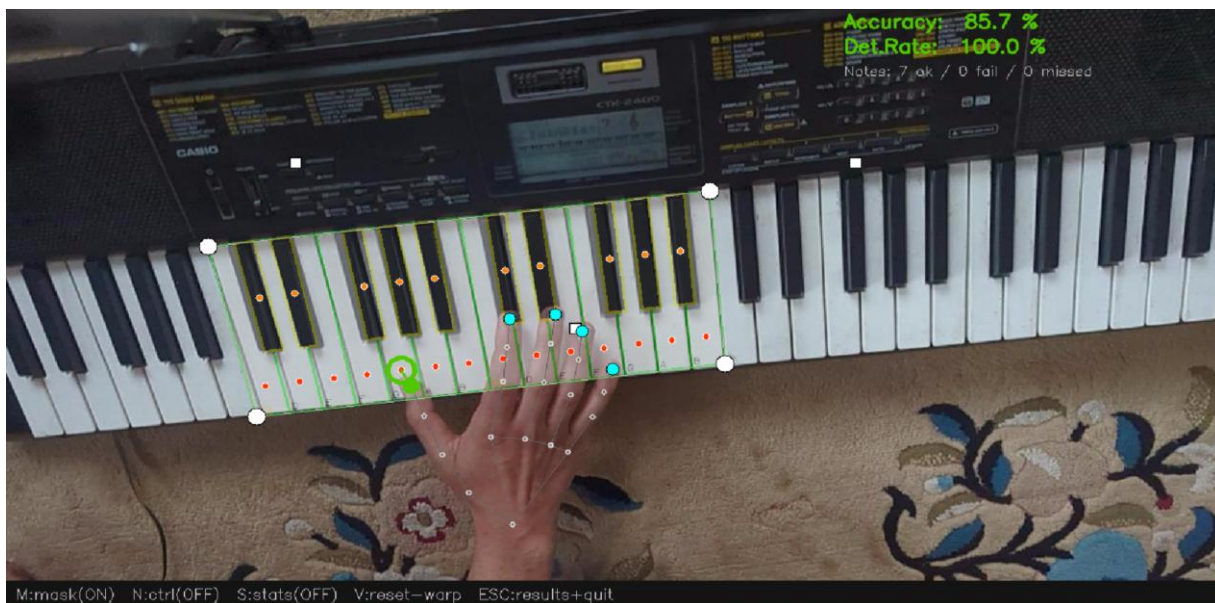
key_calibration.py running — keyboard overlay aligned to physical piano keys

Key controls:

Key	Action
Drag interior	Move the whole overlay
Drag edges	Resize key width or height
Drag corner ○	Warp perspective to correct camera angle
Scroll wheel	Add or remove a key on the right
[/]	Shift the starting MIDI note
C	Toggle centre visualisation (line / dot)
F	Toggle 180° flip to match record.py
H	Toggle help overlay
ENTER or S	Save calibration
ESC	Quit without saving

5.2 Live MPJPE Test (live_preview.py)

Opens a setup screen to select the MIDI port and camera. Once running, play the piano and MPJPE accuracy is shown live in the top-right corner. Press ESC to stop and view the final results panel.



live_preview.py main view — hand skeleton, key overlay, MPJPE readout top-right

MJMPE TEST RESULTS

MJMPE (horizontal)5.7 px (med 5.0)

Accuracy (<10 px)85.1 %

Detection rate95.7 %

Notes matched134

Detection fails (no tip on key)6

Notes missed (no hands)0

Max error15.0 px

Per-finger breakdown (L = lower keyboard half, R = upper keyboard half):

Left Hand

Thumb6.3px 76.9% (13)

Index3.6px 100.0% (12)

Middle2.5px 100.0% (12)

Ring3.4px 92.3% (13)

Pinky8.9px 60.0% (10)

Right Hand

Thumb6.5px 75.0% (8)

Index5.0px 95.0% (20)

Middle6.0px 95.2% (21)

Ring6.6px 85.7% (14)

Pinky9.7px 45.5% (11)

Press any key to close

live_preview.py results panel showing MPJPE, accuracy %, and per-finger breakdown

Key controls during test:

Key	Action
M	Toggle key mask overlay on/off
N	Toggle mask control panel
C	Toggle centre visualisation (line / dot)
S	Toggle FPS / latency stats overlay
V	Reset perspective warp to rectangle
ESC	Stop test and show results

5.3 Recording Session (record.py)

Records a full participant session. The setup screen collects participant ID, Fitzpatrick skin type, ambient lux, and hand size before recording begins. Video, MIDI, and metadata are saved to `data/raw/p###/`.

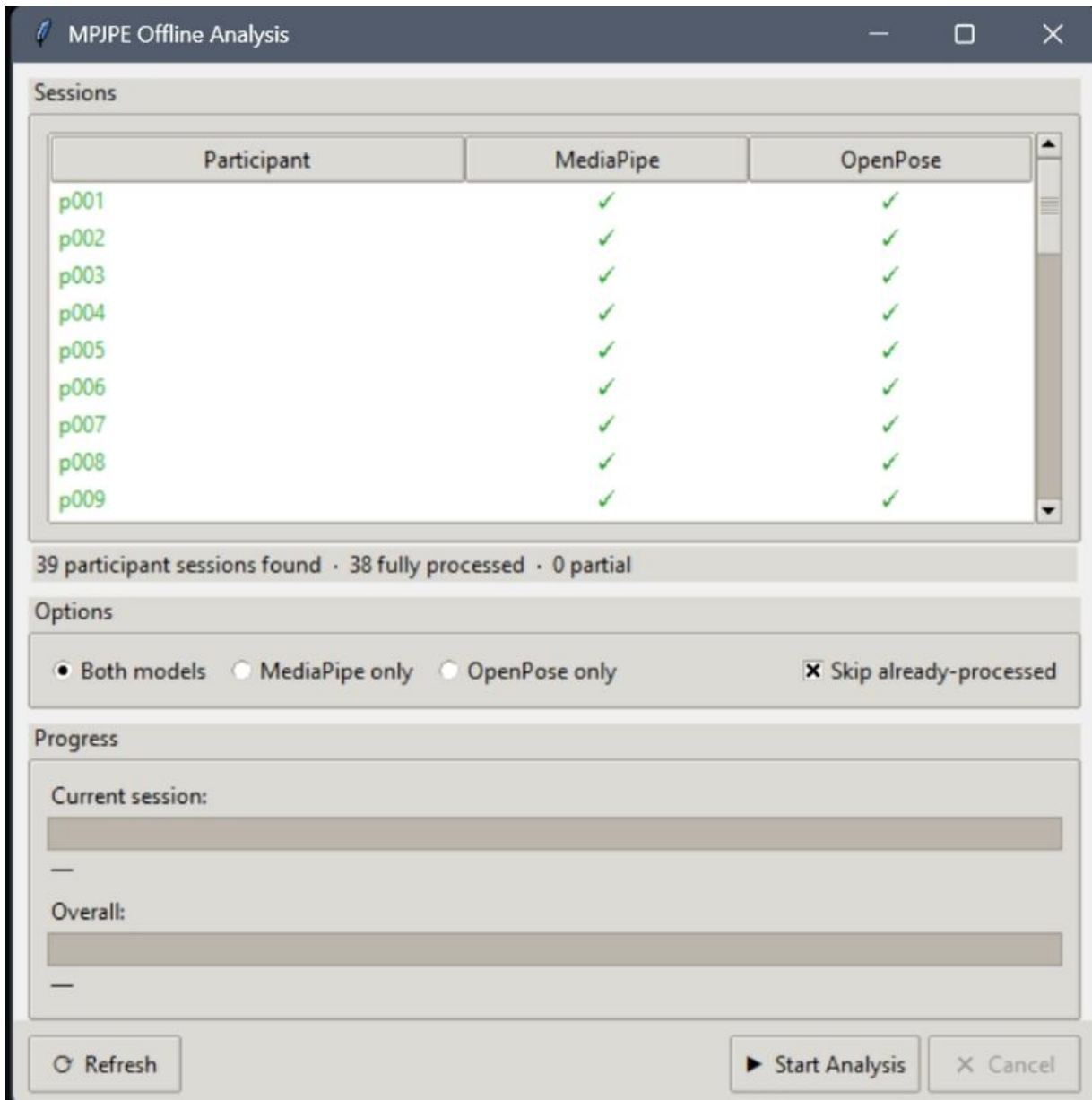


record.py setup screen — participant ID, Fitzpatrick type, lux, and hand-size fields

5.4 Offline Analysis (analyse.py)

Processes recorded sessions to compute MPJPE metrics for MediaPipe and/or OpenPose. Can be run via the GUI (double-click sessions to process) or from the command line:

```
python -m scripts.analyse # open GUI
python -m scripts.analyse data/raw/p001/ # process one session
python -m scripts.analyse data/raw/p001/ --model openpose
```

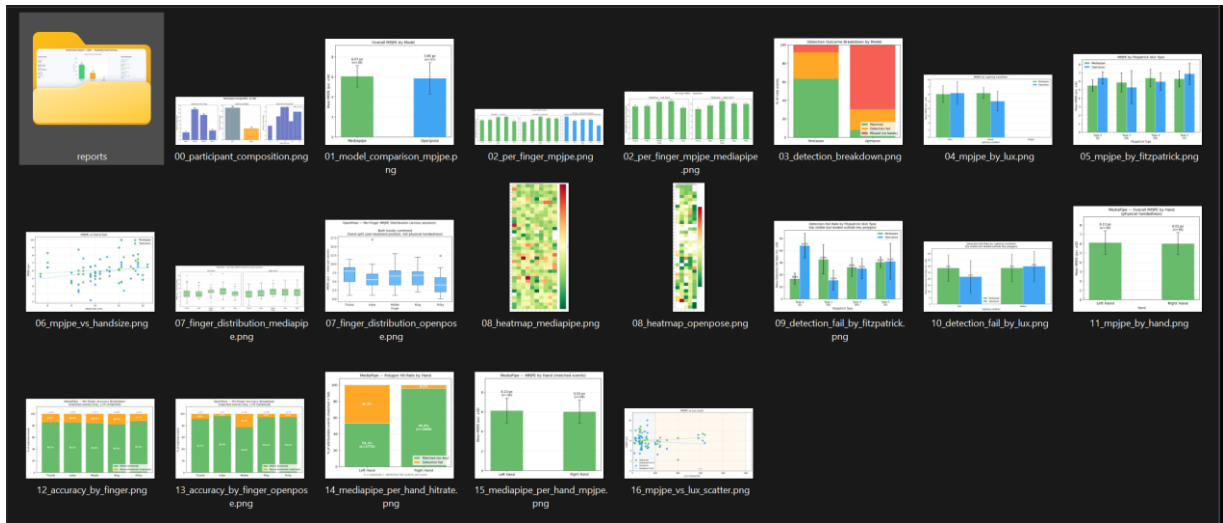


analyse.py Tkinter GUI showing session list with MediaPipe and OpenPose columns

5.5 Plot Results (plot_results.py)

Reads all results JSON files under data/processed/ and generates group-level charts and per-participant PDFs in data/plots/:

```
python -m scripts.plot_results
```



data/plots/ folder — generated PNG charts visible in file browser

6. Data Folder Structure

The following folder structure is created automatically as sessions are recorded and processed:

```
data/
├── calibration/
│   └── key_centers.json          ← saved by key_calibration.py
├── raw/
│   └── p001/
│       ├── p001.mp4             ← raw video (no overlays)
│       ├── p001_frames.csv      ← frame timestamps
│       ├── p001_midi.jsonl      ← MIDI events
│       ├── p001_midi.mid        ← standard MIDI file
│       └── p001_session.json    ← session metadata
├── processed/
│   └── p001/
│       ├── p001_mediapipe_results.json
│       └── p001_openpose_results.json
└── plots/
    ├── 01_model_comparison_mpjpe.png
    └── ...                      ← all group charts
```

7. Troubleshooting

Problem	Likely cause	Fix
pip install fails on python-rtmidi	Missing C++ compiler	Install Visual Studio Build Tools (Desktop development with C++)

ModuleNotFoundError: mediapipe	Wrong Python version or venv not active	Confirm python --version is 3.10.x and (piano_env) prefix is shown
No MIDI ports detected	Keyboard not connected or driver missing	Connect keyboard before launching; install manufacturer USB-MIDI driver
Camera shows black / wrong camera	Wrong camera index in config.py	Edit CAMERA_INDEX in scripts/config.py (try 0, 1, 2...)
caffemodel missing error	Large file not pulled from Git LFS	Run python download_openpose_model.py to re-download
All core imports OK but MediaPipe crashes	Python 3.11+ installed	Switch to Python 3.10 and recreate the virtual environment